

Zadanie 1: Kosztorys roczny

Parametry:

- **10 TB danych historycznych**
- **200 GB / miesiąc przyrost = 2.4 TB rocznie**
- **Razem: ~12.4 TB**

Usługi:

1. **Storage Account** (12.4 TB w Hot Access Tier)
2. **Azure Databricks Premium (per DBU / Cluster time)**
3. **Key Vault** (do przechowywania tajemnic)
4. **Data Factory** (do orkiestracji danych)
5. **SQL Server** (np. Azure SQL Database jako warstwa prezentacyjna lub metadane)

Service	Monthly Cost (USD)	Yearly Cost (USD)
Storage Account (Hot Tier)	228.5568	2742.6816
Azure Databricks (Premium DBU + Jobs)	1500	18000
Key Vault	15	180
Data Factory (1 pipeline daily)	100	1200
SQL Server (Azure SQL DB - Standard 100 DTU)	150	1800
Total	1993.5568	23922.6816

Zadanie 2: Delta Lake vs Apache Iceberg – porównanie.

Cecha	Delta Lake	Apache Iceberg
Dostępność	Zintegrowany z Azure Databricks	Wspierany w wielu silnikach (Spark, Trino, Flink)
Łatwość użycia	Bardzo dobra w ekosystemie Databricks	Wymaga integracji
ACID	Tak (silna obsługa transakcji)	Tak
Time Travel	Tak	Tak
Wsparcie dla schema evolution	Tak	Tak (bardziej elastyczne)
Formaty danych	Parquet (wewnętrznie)	Parquet, ORC
Integracja z Azure	Ścisła z Databricks	Wymaga więcej konfiguracji

Rekomendacje:

- Delta Lake: najlepszy wybór, gdy używasz Databricks i chcesz szybko wdrożyć solidną architekturę lakehouse z pełnym wsparciem.
- Iceberg: sprawdzi się, gdy klient ma heterogeniczne środowisko (np. Trino, Flink, Spark) i zależy mu na otwartym, wieloplatformowym formacie.

Zadanie 3: Krytyka architektury Medalionu.

Oto lista potencjalnych wad i zastrzeżeń:

1. **Złożoność operacyjna** – wymaga wielu warstw, co zwiększa nakład pracy.
2. **Opóźnienia w dostępności danych** – dane dostępne dopiero po przetworzeniu kolejnych warstw.
3. **Duplikacja danych** – te same dane są kopiowane między warstwami.
4. **Koszt przechowywania** – dane są zapisywane kilkakrotnie.
5. **Trudność w debugowaniu** – błędy mogą być propagowane przez warstwy.
6. **Utrata śledzenia zmian** – brak wersjonowania pomiędzy warstwami może utrudnić analizy.
7. **Brak elastyczności w schemacie** – rigidne podejście może ograniczać użycie elastycznych danych.
8. **Zbyt ogólny model** – nie wszystkie przypadki wymagają Gold/Silver/Bronze.
9. **Niedostosowanie do real-time** – opóźnienia i batch processing są naturalne.
10. **Trudność w monitorowaniu** – wiele warstw utrudnia monitoring i alerty.
11. **Czasochłonne przetwarzanie** – dane przechodzą przez wiele ETL.
12. **Wysoki próg wejścia dla zespołu** – trudniejsze szkolenie i wdrożenie nowych osób.
13. **Powielanie kodu transformacji** – np. podobne logiki w Silver i Gold.
14. **Problemy z synchronizacją danych** – inkrementalne odświeżenia mogą powodować niespójności.
15. **Złożone zarządzanie jakością danych** – brak jednego punktu kontroli.

16. **Brak standaryzacji narzędzi do implementacji** – zależność od platformy (np. Databricks).
17. **Nieoptymalne dla małych projektów** – overkill w prostych przypadkach.
18. **Przesadne "warstwowanie"** – może prowadzić do niepotrzebnych komplikacji.
19. **Słaba widoczność dla biznesu** – trudniej zrozumieć przepływ danych.
20. **Zarządzanie wersjami modeli i raportów** – trudniejsze śledzenie zmian biznesowych w Gold layer.